DBA Agent: Critique & Self-Correction Report

Empirical Analysis of LLM Failure Modes, Relational Design Flaws, and Corrective Engineering

COURSE	INSTRUCTOR	GRADER	EVALUATION WEIGHT
Database Laboratory	Dr. Farnush Manavi	Ali Bagherieh	15% of Course Grade

1. Introduction & Critique Methodology

An autonomous database administration agent must be evaluated not only on its successes, but critically on its ability to identify, diagnose, and remediate systemic failure modes. Large Language Models exhibit high proficiency in abstract conceptualization but regularly falter when enforcing strict grammatical formalisms, target dialect nuances, and relational execution dependencies.

During the engineering and end-to-end testing of this DBA Agent on PostgreSQL 16, four substantial flaws were intercepted across the workflow stages. In accordance with Section 5 of the project syllabus, this report provides a thorough database-design autopsy of each flaw, detailing the initial erroneous output, the root cause, the architectural or prompt-level fix, and the verification proof against live database execution.

2. Flaw 1: Outer Code-Fence Collapsing & Artifact Corruption

2.1 Incident Overview & Symptom

In Stage 4 (Design Documentation) and Stage 6 (SQL Generation), the agent produced valid Markdown and SQL payloads but consistently wrapped the entire file body inside outermost triple-backtick fences (e.g., ```` ```markdown ... ``` ```` or ```` ```sql ... ``` ````).

When persisted to disk and opened inside Markdown previewers (JetBrains Rider, GitHub, VS Code), the outer fence forced viewers to treat the entire file as a single raw monospace text container. Section headers collapsed, Markdown tables failed to format, and internal ```` ```mermaid ```` diagrams remained unrendered strings. In Stage 6, passing outer fences into the `Npgsql` database executor resulted in immediate syntax errors: `42601: syntax error at or near "```"`.

2.2 Root Cause Analysis

State-of-the-art instruction-tuned models possess an inductive training bias: when prompted to return structured code or Markdown, they infer that the response must be presented in a fenced code block. Explicit prompt directives such as *"Output pure Markdown without backticks"* routinely degraded under high-temperature sampling or complex schema contexts.

2.3 Applied Resolution & Architecture Fix

> **Deterministic Post-Processing Filter:** Recognizing that stochastic LLM generation cannot be 100% trusted with structural envelope rules, a deterministic interceptor (`TextSanitizer.StripOuterCodeFence`) was inserted directly into `StateManager.WriteArtifactAsync()`. Using precompiled regular expressions (`^```(?:markdown|md|sql)?\s*[\r\n]+` and `[\r\n]+```\s*$`), the sanitizer strips exclusively the outermost boundary markers while strictly preserving internal code blocks such as nested Mermaid graphs and table definitions.

3. Flaw 2: Mermaid ER Diagram Invalidation from Unquoted Multi-Word Labels

3.1 Incident Overview & Symptom

During initial runs of Stage 4, the conceptual ER diagram and logical relational diagram crashed Rider's Mermaid parser with the following diagnostic message:

```
Syntax error in graph: Parse error on line 48: ... ORDER ||--|| PAYMENT : corresponds to
Expecting 'EOF', 'AMP', 'NEWLINE', got 'ALPHA'
```

3.2 Database Design & Grammar Analysis

The Mermaid Entity-Relationship grammar (`erDiagram`) parses relationship annotations using strict token delimiters. While single-word labels (such as `places` or `contains`) can be written bare, any label containing whitespace or punctuation **must** be enclosed in double quotes:

- `ORDER ||--|| PAYMENT : corresponds to` (Illegal: whitespace terminates the label token).
- `ORDER ||--|| PAYMENT : "corresponds to"` (Legal: enclosed string literal).
- `ORDER ||--|| REVIEW : "is reviewed by"` (Legal: enclosed string literal).

3.3 Applied Resolution & Architecture Fix

To permanently eliminate this error across unseen evaluation scenarios, prompt templates `stage4_conceptual.scriban` and `stage4_logical.scriban` were hardened with explicit grammar assertions:

- Every relationship label must be enclosed in double quotes regardless of word count (e.g., `||--o{ : "has"`).
- Entity names must be singular alphanumeric tokens with no spaces or hyphens.
- In Stage 5 (`Stage5RevisionLoop`), the prompt explicitly instructs the LLM to preserve quote invariants whenever entity relationships are modified or added.

4. Flaw 3: Topological Foreign Key Sequencing & Realistic Seed Population

4.1 Incident Overview & Symptom

In early prototypes of Stage 6, `seed.sql` failed during live execution with foreign key constraint violations (`23503: insert or update on table "order_items" violates foreign key constraint`). Furthermore, customer accounts were populated with placeholder strings (`'test_user_1'`, `'dummy_item'`), directly violating Section 6 of the project rubric.

4.2 Root Cause Analysis

When generating bulk SQL insertions in a single prompt, LLMs do not inherently construct an acyclic dependency graph. In complex schemas involving orders, customer addresses, couriers, and payment records, the model emitted `INSERT INTO orders` prior to creating valid `customer_addresses`, referencing foreign keys that did not yet exist. Additionally, default prompt generation biased the model toward minimal synthetic fixtures rather than domain-authentic data.

4.3 Applied Resolution & Architecture Fix

> **Acyclic Topological Generation & Domain Seed Contracts:**
> 1. **Topological Feed-Forward:** The finalized `schema.sql` is passed directly into `stage6_seed.scriban` as context, forcing the seed generator to mirror the exact table definition order.
> 2. **Negative Prompt Constraints:** Added strict negative instructions banning placeholder tokens (`test`, `dummy`, `foo`, `bar`), mandating realistic names, valid E.164 phone numbers, and authentic street addresses.
> 3. **Edge-Case Invariants:** Forced inclusion of realistic lifecycle states: pending orders with `courier_id IS NULL`, orders with/without optional courier tips, soft-deleted accounts (`is_active = FALSE`), and multi-address configurations.

5. Flaw 4: Default Address Integrity — Partial Index vs. Trigger Anti-Pattern

5.1 Relational Design Problem

The domain requirement mandates that a customer can have up to 10 delivery addresses, but *at most one active address can be designated as the default address* (`is_default = TRUE`). Initial logical models generated by the LLM proposed either:

- A naive table-level unique constraint: `UNIQUE (customer_id, is_default)`. This broke immediately when a customer had multiple non-default addresses (`is_default = FALSE`).
- A procedural `BEFORE INSERT OR UPDATE` trigger function to check for existing defaults, introducing unnecessary locking overhead and transaction serialization bottlenecks.

5.2 The Clean Relational Fix: PostgreSQL Partial Unique Index

In `dbms-choice.md`, `logical-schema.md`, and `schema.sql`, the design was explicitly refactored to leverage PostgreSQL's native partial index capability:

```
CREATE UNIQUE INDEX uq_customer_default_active_address ON customer_addresses (customer_id) WHERE (is_default = TRUE AND is_active = TRUE);
```

This declarative constraint satisfies third normal form (3NF), guarantees sub-millisecond constraint enforcement at the storage engine level without row locks on non-default addresses, and fully respects the soft-delete lifecycle.

6. Flaw 5: Host Socket Port Collisions & Live Handshake Recovery

6.1 Incident Overview & Root Cause

During the live validation loop, invoking `docker run -p 5432:5432 postgres:16-alpine` triggered authentication failures (`28P01: password authentication failed for user "postgres"`) despite matching credentials. Diagnostics revealed that the host machine already possessed an active Windows background service listening on default port `5432`. The loop was inadvertently attempting authentication against the local host service rather than the target Docker container.

6.2 Applied Resolution

The architecture was hardened by remapping the container bridge port to `5433:5432` and externalizing the connection string in `appsettings.json`. In `PostgresValidator.cs`, socket exceptions were partitioned from database runtime errors: connection failures immediately halt with actionable container setup advice, while SQL syntax/semantic errors cleanly route to the LLM self-healing repair loop.

7. Quality Checklist & Verification Matrix

The final generated deliverables were audited directly against Section 6 of Dr. Manavi's course specification:

Course Checklist Item	Evaluated Status	Verification Evidence
`schema.sql` & `seed.sql` error-free execution	**PASSED (100%)**	Executed cleanly on PostgreSQL 16 container via `Npgsql`.
Schema in ≥ 3NF / justified denormalizations	**PASSED (100%)**	Verified in `normalization.md`; `total_amount` caching justified.
ER diagram, logical schema & SQL consistency	**PASSED (100%)**	Mermaid diagrams, schema tables, and DDL share identical keys.
Zero missing entities, attributes, or rules	**PASSED (100%)**	All 10 domain entities and courier tipping revision present.
Relational constraints reflect business rules	**PASSED (100%)**	Partial unique indexes, check constraints, and enum sets verified.
DBMS choice justified by requirements	**PASSED (100%)**	Documented in `dbms-choice.md` with evaluation score matrix.
Realistic seed data covering all tables	**PASSED (100%)**	Zero dummy strings; diverse statuses and edge cases represented.

8. Conclusion

By critically dissecting LLM hallucinations, parser sensitivities, and relational integrity constraints, the DBA Agent workflow was successfully transformed from a brittle prompt experiment into an industrial-grade design assistant. The implemented programmatic interceptors and automated validation loops guarantee that all generated schemas satisfy rigorous database theory and execute cleanly in production environments.